

```

import socket
import threading
import time
import json
import hashlib

# --- SYMBIONT OMEGA HYBRID v15.6-OMEGA ---
PORT = 9000
BROADCAST_PORT = 9001

class SymbiontNode:
    def __init__(self):
        self.node_id = "symbiont-" +
hashlib.md5(str(time.time()).encode()).hexdigest()[:16]
        self.memory = [] # Блоки цепочки
        self.quests = [] # Общие задачи
        self.peers = {} # IP -> last_seen
        self.running = True

        # Начальный блок (Genesis)
        self.add_block("Genesis Block initialized")

    def add_block(self, data):
        block_id = len(self.memory)
        block = {"id": block_id, "data": data, "timestamp":
time.time()}
        self.memory.append(block)
        return block

    def start_servers(self):
        # Сервер для прямых TCP-соединений (обмен данными и квестами)
        tcp_thread = threading.Thread(target=self._tcp_server,
daemon=True)
        tcp_thread.start()

        # Сервер для автоматического поиска в локальной сети (UDP
Broadcast)
        udp_thread =
threading.Thread(target=self._udp_broadcast_listener, daemon=True)
        udp_thread.start()

        # Поток для рассылки своего присутствия
        beacon_thread = threading.Thread(target=self._udp_beacon,
daemon=True)
        beacon_thread.start()

    def _tcp_server(self):
        server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

```

```

server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
server.bind(('0.0.0.0', PORT))
server.listen(5)
while self.running:
    try:
        conn, addr = server.accept()
        threading.Thread(target=self._handle_peer, args=(conn,
addr), daemon=True).start()
    except:
        break

def _handle_peer(self, conn, addr):
    try:
        data = conn.recv(4096)
        if data:
            message = json.loads(data.decode('utf-8'))
            self._process_message(message, conn, addr[0])
    except:
        pass
    finally:
        conn.close()

def _process_message(self, msg, conn, ip):
    m_type = msg.get("type")
    if m_type == "SYNC_REQ":
        # Отправляем свои квесты и память в ответ
        response = {
            "type": "SYNC_RES",
            "node_id": self.node_id,
            "memory": self.memory,
            "quests": self.quests
        }
        conn.sendall(json.dumps(response).encode('utf-8'))
        self.peers[ip] = time.time()
    elif m_type == "NEW_REQUEST":
        quest = msg.get("quest")
        if quest not in self.quests:
            self.quests.append(quest)
            print(f"\n[Mesh] Получен новый квест: {quest}")

def _udp_beacon(self):
    # Рассылаем UDP-сигнал каждые 5 секунд, чтобы нас нашли
    s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    s.setsockopt(socket.SOL_SOCKET, socket.SO_BROADCAST, 1)
    while self.running:
        try:
            message = json.dumps({"type": "DISCOVER", "node_id":
self.node_id, "port": PORT})

```

```

        s.sendto(message.encode('utf-8'), ('<broadcast>',
BROADCAST_PORT))
    except:
        pass
    time.sleep(5)

def _udp_broadcast_listener(self):
    # Слушаем сигналы от других устройств в сети
    s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    try:
        s.bind(('0.0.0.0', BROADCAST_PORT))
    except:
        return

    while self.running:
        try:
            data, addr = s.recvfrom(1024)
            message = json.loads(data.decode('utf-8'))
            ip = addr[0]
            if message.get("node_id") != self.node_id:
                if ip not in self.peers:
                    self.peers[ip] = time.time()
                    # Автоматически синхронизируемся при
обнаружении нового соседа
                    threading.Thread(target=self.sync_with_peer,
args=(ip,), daemon=True).start()
        except:
            break

def sync_with_peer(self, ip):
    try:
        s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        s.settimeout(3)
        s.connect((ip, PORT))
        req = {"type": "SYNC_REQ", "node_id": self.node_id}
        s.sendall(json.dumps(req).encode('utf-8'))

        data = s.recv(16384)
        if data:
            res = json.loads(data.decode('utf-8'))
            # Сливаем чужие квесты
            for q in res.get("quests", []):
                if q not in self.quests:
                    self.quests.append(q)

        s.close()
    except:
        pass

```

```

def broadcast_quest(self, quest):
    for ip in list(self.peers.keys()):
        try:
            s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
            s.settimeout(2)
            s.connect((ip, PORT))
            msg = {"type": "NEW_QUEST", "node_id": self.node_id,
"quest": quest}
            s.sendall(json.dumps(msg).encode('utf-8'))
            s.close()
        except:
            pass

# --- Интерфейс командной строки ---
node = SymbiontNode()
node.start_servers()

print("=====")
print(f"SYMBIOSIS OMEGA HYBRID v15.6-OMEGA")
print(f"Node ID: {node.node_id}")
print("Автоматический поиск пиров в сети запущен...")
print("Команды: status | memory | quest <title> | peers | ask <text> | exit")
print("=====")

whileType = True
while whileType:
    try:
        cmd = input(f"[{list(node.peers.keys())}] > ").strip()
        if not cmd:
            continue

        if cmd == "exit":
            node.running = False
            break
        elif cmd == "status":
            print(f"Memory blocks: {len(node.memory)} | Direct peers: {len(node.peers)}")
        elif cmd == "memory":
            for b in node.memory:
                print(f"[{b['id']}] {b['data']}")
        elif cmd == "peers":
            print("Active peers IPs:", list(node.peers.keys()))
        elif cmd.startswith("quest "):
            q_title = cmd[6:]
            node.requests.append(q_title)
            node.broadcast_quest(q_title)

```

```
        print(f"[Квест создан и отправлен в сеть]: {q_title}")
    elif cmd == "quest":
        print("Список квестов:", node.quests)
    elif cmd.startswith("ask "):
        text = cmd[4:]
        node.add_block(text)
        print(f"[Память обновлена]: {text}")
    else:
        node.add_block(cmd)
        print(f"[Записано в локальную сеть]: {cmd}")
except KeyboardInterrupt:
    node.running = False
    break
```